Module 8
Assignment 8B:
Rebuild Your Inventory with Laravel Eloquent

Course: CS 85 - PHP Programming
Instructor: Vicky Seno

Project Overview

For this assignment, I rebuilt my inventory application using Laravel migrations, an Eloquent model, a controller, routing, and a Blade template. The original inventory from Module 4B used direct database access, while this version uses Laravel's ORM tools to create a cleaner and more organized workflow.

Module 8B is implemented as an isolated feature inside my main CS85 Laravel application. It shares the main Laravel installation and dependencies, but it has its own database connection, migration directory, controller, Blade view, tests, and documentation. This avoids creating a second Laravel application inside the existing project.

GitHub Repository URL: https://github.com/sergehall/cs85-php-programming

Assignment Documentation: https://github.com/sergehall/cs85-php-programming/blob/main/assignments/module8b/README.md

Inventory Blade Template: https://github.com/sergehall/cs85-php-programming/blob/main/resources/views/assignments/module8b/inventory.blade.php

Test URL: http://localhost:8000/inventory

Step 1: Database Configuration

I configured Laravel to connect to the MySQL database named inventory_db. MySQL runs inside the cs85-mysql Docker container and is available to Laravel through port 3307.

The local credentials are stored in the ignored .env file. The sanitized .env.example contains placeholders so that no real database password is committed to GitHub.

```
MODULE8B_DB_HOST=127.0.0.1
MODULE8B_DB_PORT=3307
MODULE8B_DB_DATABASE=inventory_db
MODULE8B_DB_USERNAME=module8b
MODULE8B_DB_PASSWORD=replace_with_local_password
```

Step 2: Create the Migration

I created a Laravel migration that rebuilds the items table. The migration defines the item name, category, quantity, purchase date, timestamps, and primary key.

```
Schema::connection('module8b')->create('items', function (Blueprint $table): void {
    $table->id();
    $table->string('item_name');
    $table->string('category')->nullable();
    $table->integer('quantity')->default(0);
    $table->date('purchase_date')->nullable();
    $table->timestamps();
});
```

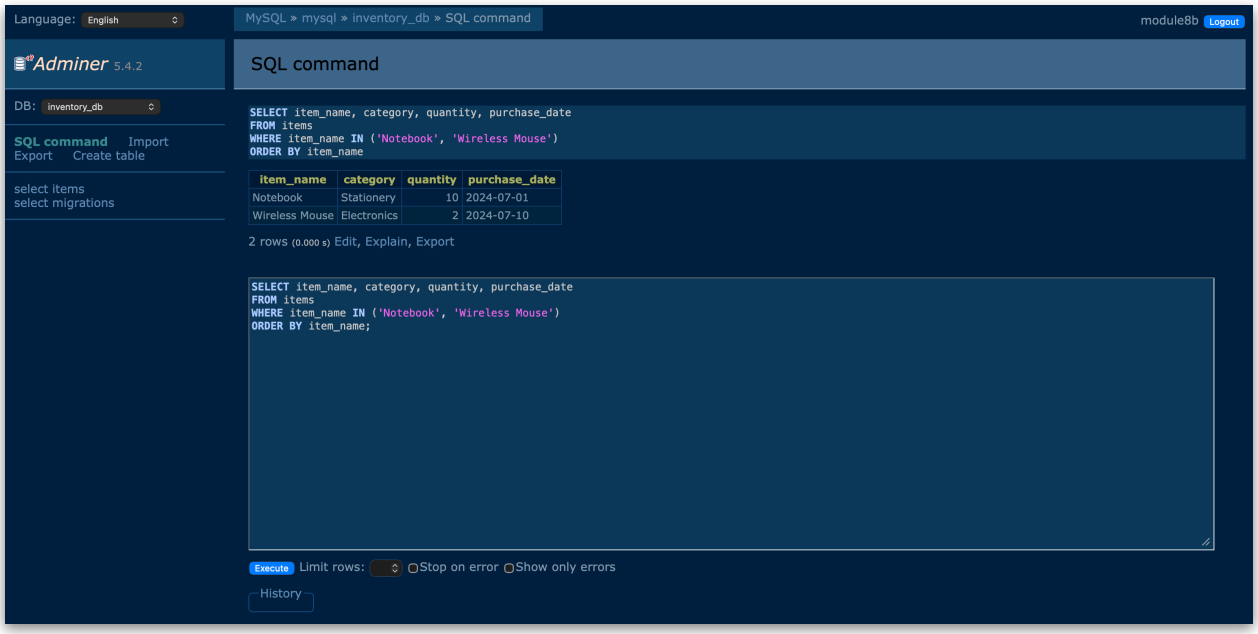
I ran the migration using the dedicated Module 8B connection and migration directory:

```
php artisan migrate \
  --database=module8b \
  --path=database/migrations/module8b
```



Figure 1. Migration status.

Step 3: Create the Eloquent Model



I created the App\Models\Item model to connect Laravel objects to rows in the items table. The $fillable property identifies the fields that can be assigned using Item::create().

```
protected $fillable = [
    'item_name',
    'category',
    'quantity',
    'purchase_date',
];
```

The model uses the separate module8b database connection. I also added casts so that quantity is handled as an integer and purchase_date is handled as a Laravel date object.

Step 4: Insert Sample Data with Tinker

I opened Laravel Tinker from the project root:

```
php artisan tinker
```

I inserted the two required inventory records using the Eloquent Item::create() method:

```
App\Models\Item::create([
    'item_name' => 'Notebook',
    'category' => 'Stationery',
    'quantity' => 10,
    'purchase_date' => '2024-07-01',
]);

App\Models\Item::create([
    'item_name' => 'Wireless Mouse',
    'category' => 'Electronics',
```

```
    'quantity' => 2,
    'purchase_date' => '2024-07-10',
]);
```

These records were successfully saved in MySQL. I verified them through Eloquent and Adminer.

Figure 2. The required Notebook and Wireless Mouse records stored in the MySQL items table.

Step 5: Create the Controller

I created Module8bInventoryController to retrieve the inventory records from MySQL through the Item model.

The controller starts the query with:

```
$items = Item::query()->get();
```

My completed controller also supports searching, category filtering, minimum-quantity filtering, and sorting. This demonstrates that Eloquent queries can be expanded without manually building raw SQL statements.

The controller passes the resulting collection to the Blade template:

```
return view('assignments.module8b.inventory', [
    'items' => $items,
]);
```

Step 6: Define the Route

I added a route that connects the required /inventory URL to the controller:

```
Route::get('/inventory', [Module8bInventoryController::class, 'index'])
    ->name('inventory.index');
```

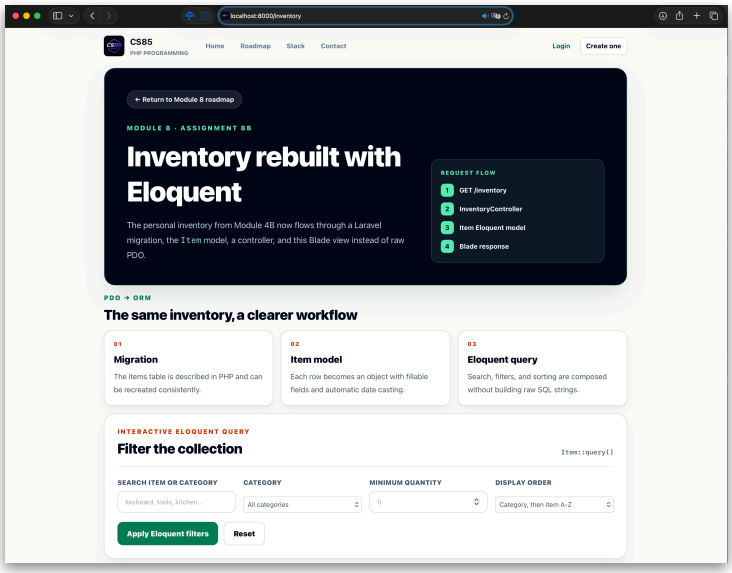
The named route allows the application to generate links using route('inventory.index') instead of manually repeating the URL.

Step 7: Display the Inventory with Blade

I created a Blade template that receives the Eloquent collection from the controller. The template uses @foreach to display each Item object.

```
@foreach ($items as $item)
    <tr>
        <td>{{ $item->item_name }}</td>
        <td>{{ $item->category ?? 'Uncategorized' }}</td>
        <td>{{ $item->quantity }}</td>
        <td>{{ $item->purchase_date?->format('M j, Y') }}</td>
    </tr>
@endforeach
```

The completed page displays the item name, category, quantity, and purchase date. I also added interactive search, filtering, and sorting controls that create new Eloquent queries through URL parameters.



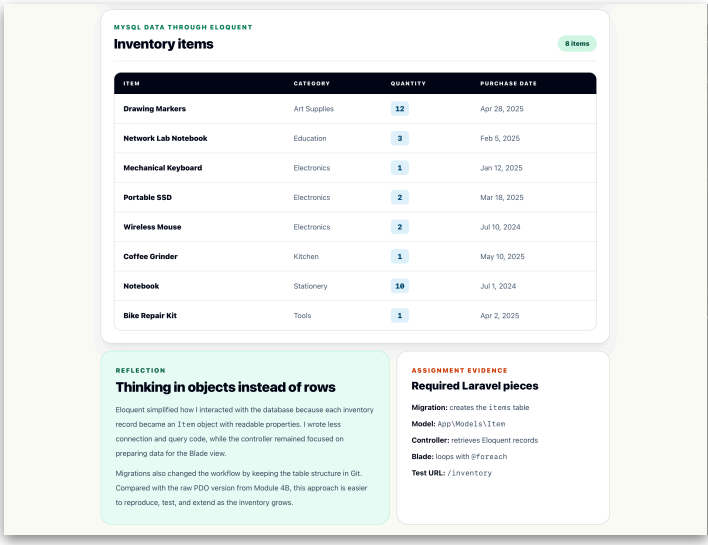


Figure 3. The completed Laravel Eloquent inventory page at http://localhost:8000/inventory.

Reflection

Eloquent simplified how I interacted with the database because each inventory record became an Item object with readable properties. Instead of manually opening a PDO connection and writing SQL for every request, I could use methods such as Item::query(), where(), orderBy(), and get().

This assignment also helped me understand the relationship between migrations and models. The migration describes the database table, while the Eloquent model represents the data in PHP. Keeping the table structure in a migration makes the database easier to recreate, review, and maintain through Git.

Compared with my earlier inventory assignment, this approach required less database connection code and kept each part of the application organized. The route handles the URL, the controller prepares the data, the model communicates with MySQL, and the Blade template displays the result. I think this is a more modern and scalable way to build database applications.

Reflection Included in the Blade Template

{{--
Reflection:
Eloquent simplified how I interacted with the database because each inventory record became an Item object instead of a raw SQL result. It helped me write less database code and keep the controller focused on preparing data for the Blade view. Migrations also made the table structure repeatable and easier to maintain, making this a more modern and scalable database workflow.
—}}

Testing and Final Result

I added feature tests for the Module 8B inventory workflow. The tests verify the route, Eloquent data retrieval, Blade output, search, filtering, sorting, and handling of invalid query parameters.

Final test result:

7 tests passed
45 assertions passed
The application is available at: http://localhost:8000/inventory